

Reinforcement Learning

Monte Carlo Methods

Min-hwan Oh

Seoul National University

- So far in dynamic programming, we assumed the complete model (dynamics of MDPs $p(s', r|s, a)$) is known to the agent...

- So far in dynamic programming, we assumed the complete model (dynamics of MDPs $p(s', r|s, a)$) is known to the agent...
- In real life, such knowledge is rarely available.

Monte Carlo Methods

- Learn from **experience**
 - ▶ What is experience?

Monte Carlo Methods

- Learn from **experience**
 - ▶ What is experience? Sample sequences of $\{s, a, r, s'\}$

Monte Carlo Methods

- Learn from **experience**
 - ▶ What is experience? Sample sequences of $\{s, a, r, s'\}$
- Episodic, undiscounted

Monte Carlo Methods

- Learn from **experience**
 - ▶ What is experience? Sample sequences of $\{s, a, r, s'\}$
- Episodic, undiscounted
- Incremental updates in an episode-by-episode sense

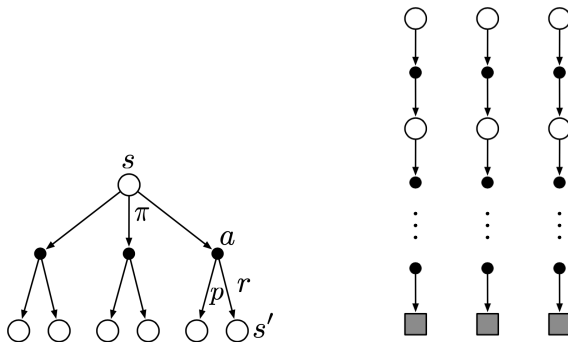
Monte Carlo Methods

- Learn from **experience**
 - ▶ What is experience? Sample sequences of $\{s, a, r, s'\}$
- Episodic, undiscounted
- Incremental updates in an episode-by-episode sense
- Sample and average **returns** for each state-action pair
 - ▶ similar to the bandit methods (week 1)

Relationship to Dynamic Programming (DP)

- Monte Carlo (MC) doesn't need a (full) model of MDP
 - ▶ Can learn from actual or simulated experience
- DP takes advantage of a full model
 - ▶ Doesn't need any experience
- MC's expense independent of number of states
- No **bootstrapping** in MC
 - ▶ Not harmed by violations of Markov property

Bootstrap vs Roll-outs



- (Left) bellman backup bootstrap in DP vs (right) MC roll-outs

Monte Carlo Prediction

- Learn state-value function for a given policy

Monte Carlo Prediction

- Learn state-value function for a given policy (policy evaluation)

Monte Carlo Prediction

- Learn state-value function for a given policy (policy evaluation)
- Estimate value of a state from experience
 - ▶ Average the returns observed after visits to that state
- First-visit MC method: estimates $v_{\pi}(s)$ as the average of the returns following first visits to s

“First-visit” Monte Carlo Prediction

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

First-visit vs. Every-visit

- Every-visit MC method: averages the returns following all visits to s
- Both first-visit MC and every-visit MC converge to $v_\pi(s)$ as the number of visits (or first visits) to s goes to infinity
- First-visit MC case:
 - ▶ Each return is an i.i.d. estimate of $v_\pi(s)$ with finite variance.
 - ▶ By the law of large numbers, averages converge to their expected value.
- Every-visit MC is less straightforward
 - ▶ But extends more naturally to function approximation and eligibility traces (Chapters 9 and 12)

Monte Carlo Prediction of Action Values

- If model is known, state-values are sufficient to determine a policy
 - ▶ simply looks ahead one step and chooses whichever action leads to the best combination of reward and next state (in DP)

Monte Carlo Prediction of Action Values

- If model is known, state-values are sufficient to determine a policy
 - ▶ simply looks ahead one step and chooses whichever action leads to the best combination of reward and next state (in DP)
- What if model is **not** known?

Monte Carlo Prediction of Action Values

- If model is known, state-values are sufficient to determine a policy
 - ▶ simply looks ahead one step and chooses whichever action leads to the best combination of reward and next state (in DP)
- What if model is **not** known?
 - ▶ Must explicitly estimate the value of each action in order for the values to be useful in suggesting a policy

Monte Carlo Prediction of Action Values

- If model is known, state-values are sufficient to determine a policy
 - ▶ simply looks ahead one step and chooses whichever action leads to the best combination of reward and next state (in DP)
- What if model is **not** known?
 - ▶ Must explicitly estimate the value of each action in order for the values to be useful in suggesting a policy
 - ▶ Need to estimate $q_{\pi}(s, a)$ instead of $v_{\pi}(s)$!

Monte Carlo Prediction of Action Values

- If model is known, state-values are sufficient to determine a policy
 - ▶ simply looks ahead one step and chooses whichever action leads to the best combination of reward and next state (in DP)
- What if model is **not** known?
 - ▶ Must explicitly estimate the value of each action in order for the values to be useful in suggesting a policy
 - ▶ Need to estimate $q_{\pi}(s, a)$ instead of $v_{\pi}(s)$!
- MC prediction of action value $q_{\pi}(s, a)$
 - ▶ Estimate the expected return when starting in state s , taking action a , and thereafter following policy π

Challenge of MC Prediction of $q_{\pi}(s, a)$

- Many state–action pairs may never be visited: problematic because
 - ▶ Need to choose among the actions available in each state
 - ▶ To compare alternatives, need to estimate the value of all the actions from each state
- Exploration is needed!
 - ▶ Exploring starts: every pair has a nonzero probability of being selected at the beginning of episodes
 - ▶ Stochastic policies with nonzero probability of selecting all actions in each state

Monte Carlo Control

- Follow the same pattern as **generalized policy iteration (GPI)**
- Iterate between policy evaluation and improvement

$$\pi_0 \xrightarrow{E} q_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} q_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \cdots \xrightarrow{I} \pi_* \xrightarrow{E} q_{\pi_*}$$

- ▶ Policy evaluation E , policy improvement I

Monte Carlo Control

- Follow the same pattern as **generalized policy iteration (GPI)**
- Iterate between policy evaluation and improvement

$$\pi_0 \xrightarrow{E} q_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} q_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \cdots \xrightarrow{I} \pi_* \xrightarrow{E} q_{\pi_*}$$

- ▶ Policy evaluation E , policy improvement I
- What are differences from GPI?
 - ▶ Episodic updates for evaluations
 - ▶ Work with q_π instead of v_π
 - ▶ Policy improvement: $\pi(s) := \arg \max_a q(s, a)$

Policy Improvement Guarantees

- Policy improvement is done by making the policy greedy with respect to the current value function for all $s \in \mathcal{S}$

$$\pi_{k+1}(s) := \arg \max_a q_{\pi_k}(s, a)$$

- Policy improvement theorem then applies to π_k and π_{k+1}

$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \arg \max_a q_{\pi_k}(s, a)) \\ &= \max_a q_{\pi_k}(s, a) \\ &\geq q_{\pi_k}(s, \pi_k(s)) \\ &\geq v_{\pi_k}(s) \end{aligned}$$

- Assures that each π_{k+1} is better than π_k , or just as good as π_k
- MC can find optimal policies given only sample episodes and no other knowledge of the environment's dynamics.

Monte Carlo with Exploring Starts

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0

Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$

Monte Carlo Control without Exploring Starts

- Exploring starts: unlikely in practice...

Monte Carlo Control without Exploring Starts

- Exploring starts: unlikely in practice...
- Consider **soft** policies
 - ▶ $\pi(a|s) > 0$ for all $s \in \mathcal{S}$ and all $a \in \mathcal{A}(s)$
 - ▶ Gradually shifted closer and closer to a deterministic optimal policy

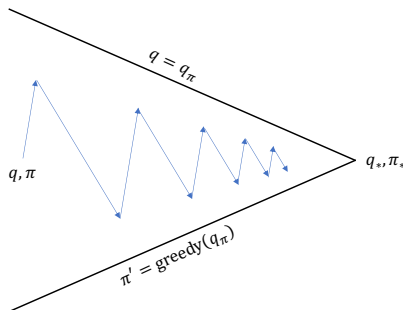
Monte Carlo Control without Exploring Starts

- Exploring starts: unlikely in practice...
- Consider **soft** policies
 - ▶ $\pi(a|s) > 0$ for all $s \in \mathcal{S}$ and all $a \in \mathcal{A}(s)$
 - ▶ Gradually shifted closer and closer to a deterministic optimal policy
- ϵ -greedy policy: explore with ϵ probability

Monte Carlo Control without Exploring Starts

- Exploring starts: unlikely in practice...
- Consider **soft** policies
 - ▶ $\pi(a|s) > 0$ for all $s \in \mathcal{S}$ and all $a \in \mathcal{A}(s)$
 - ▶ Gradually shifted closer and closer to a deterministic optimal policy
- ϵ -greedy policy: explore with ϵ probability
 - ▶ All non-greedy action with probability $\frac{\epsilon}{|\mathcal{A}(s)|}$
 - ▶ Greedy action with probability $1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|}$

Monte Carlo Control and GPI



- Overall idea of (on-policy) Monte Carlo control is still that of GPI.
 - ▶ GPI does not require that the policy be taken all the way to a greedy policy, only that it be moved toward a greedy policy.
 - ▶ Any ϵ -soft policy, π , any ϵ -greedy policy with respect to q_π is guaranteed to be better than or equal to π .

Monte Carlo with ϵ -soft policies

On-policy first-visit MC control (for ϵ -soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\epsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ϵ -soft policy

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \epsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

Today

- We learned sample-based RL methods
- On-policy MC methods
 - ▶ Prediction and Control using exploring starts and ϵ -soft policy
- Next class, we will learn “off-policy” methods.

Next class: Off-policy Monte Carlo Prediction/Control
(Continue: Chapter 5 of Sutton & Barto)